

고급파이썬프로그래밍및자동화 최종보고서

전자·정보공학부 (컴퓨터공학)

202551131 홍사강

1. 기존 코드 소개

해당 코드는 SAM3 모델을 대상으로, 2D 요리 영상에서의 일관성 있는 재료 식별, 분할 가능성을 파악하기 위한 실험 코드이다. 이 때, "일관성 있는" 재료 식별, 분할이란 크게 다음 두 가지 조건을 만족시켜야 한다.

- 동일 개념 복수 인스턴스에 대해 구분해서 segmentation, tracking
- 재료의 상태가 변했을 때도 해당 인스턴스에 대한 추적 유지 ex) 당근이 갈리거나, 잘림

이러한 조건의 정량 평가를 위한 데이터셋이 존재하지 않으므로, 본 실험은 정성 평가로 수행되었다. 이를 위해 실험 코드는 요리 영상들이 저장된 폴더 경로, 분할을 위한 프롬프트 목록을 받아 SAM3로 분할을 수행한 후, 그 결과를 인스턴스 별 색깔 마스크로 시각화한 요리 영상들로 저장한다. 기존의 실험 수행에서 긴 시간 소요와, 빈번한 OOM 문제로 인한 실험 실패가 존재했기에, 해당 문제를 개선하고자 해당 코드를 대상으로 선정하였다.

2. 기존 코드의 문제점

2.1 run_video_with_prompts()

2.1.1 함수의 책임이 과도하게 섞인 문제

기존 코드의 run_video_with_prompts() 함수는 다음과 같은 역할을 수행한다.

- 입력 영상의 프레임을 읽어온다.
- 출력 영상의 fps를 결정한다.
- 프롬프트별 출력 파일명을 생성한다.
- 기존 출력 파일 존재 여부를 검사한다.
- SAM3 세션을 시작, reset, close 한다.
- 프롬프트를 predictor에 전달한다.
- 결과 영상을 저장한다.
- OOM 및 일반 예외를 처리하고 실패한 프롬프트 목록을 수집한다.
- CUDA 메모리 정리 함수를 호출한다.

이는 단일책임원칙에 위배되며, 모호한 역할 구분으로 코드의 가독성과 유지보수성을 떨어트린다.

2.1.2 SAM3 세션 및 CUDA 정리 책임이 추론 흐름에 직접 결합된 문제

위 2.1.1의 내용에 이어서, 기존 코드는 단일 함수 안에 SAM3 세션 관리 및 CUDA 메모리 정리 로직이 예측 및 시각화 로직과 함께 함수 안에 try-except-finally 구조로 직접 관리한다. 이는 context manager와 자원 해제 캡슐화 관점에서 분리 대상이며, 세션 관리 책임을 별도 객체나 context manager로 이동시킬 필요가 있다.

2.1.3 진행 상황 출력과 실패 보고가 추론 흐름에 직접 결합된 문제

기존 코드는 [DONE], [ERROR], [REPORT]와 같은 진행 상황 출력과 실패 조합 보고를 예측 수행 흐름 안에서 직접 처리한다. 이는 logging decorator 또는 실행 정책 분리 관점에서, 핵심 추론 함수가 예측만 담당하지 못하고 관찰과 보고 책임까지 함께 갖는 구조이므로 분리가 필요하다.

2.1.4 설정값과 런타임 상태의 경계가 불명확한 문제

기존 코드는 출력 경로, suffix, fps_out, alpha, skip 여부 등과 같은 실행 설정값을 개별 인자로 직접 전달한다. 동시에 함수 내부에서는 session_id, failed_prompts와 같은 런타임 상태를 함께 관리한다. 이는 실행 과정에서 변하지 않는 설정과 실행 과정 중에 변화하는 상태의 경계가 불명확한 구조로 개선이 필요하다.

2.1.5 중복 영상 검사의 비효율적인 위치

기존 코드는 다음과 같은 흐름으로 동작한다.

- 지정된 경로에 존재하는 모든 영상의 이름을 리스트로 받아온다.
- 개별 영상을 순회하면서 다음 과정을 반복한다.
- 시각화를 위한 프레임을 읽어온다.
- 주어진 프롬프트 리스트를 순회하면서 다음을 수행한다.
- 영상 이름과 프롬프트로 출력 영상 이름을 생성한다.
- 해당 이름의 파일이 이미 존재할 경우, 예측 및 출력을 건너뛰고 다음 조합으로 넘어간다.
- 존재하지 않을 경우, 예측 및 시각화를 수행한다.

위 흐름에 따라, 기존 코드는 원본 영상의 프레임을 읽어온 후에 실제 예측 및 시각화 여부를 판단한다. 따라서 읽어온 영상에 대해 실제 예측이 일어나지 않을 경우 비효율적인 실행이 이뤄지며, 코드의 실행 시간이 불필요하게 증가한다.

2.1.6 전체 프레임 및 예측 결과를 한 번에 저장하는 eager evaluation 구조

기존 코드는 다음과 같이 두 군데에서 순차적으로 처리할 수 있는 정보를 한 번에 저장하여 메모리를 비효율적으로 사용한다.

- `read_video_frames_rgb()` 함수에서 입력 영상의 모든 프레임을 리스트로 저장한다.
- `handle_stream_request()`로부터 받은 프레임별 예측 결과를 딕셔너리로 저장한다.
- 위에서 저장한 리스트와 딕셔너리를 `save_masklet_video()` 함수에 전달하여 시각화 영상을 저장한다.

이는 수업에서 다룬 `iterator`, `generator`, `lazy evaluation` 관점에서 비효율적인 `eager evaluation` 구조에 해당한다. 특히, 영상의 길이가 길거나, 해상도가 높을수록, 또 예측된 인스턴스가 많을수록 전체 프레임 리스트와 전체 예측 결과 딕셔너리가 동시에 메모리에 유지됨으로써 발생하는 `peak memory` 사용량이 증가한다.

2.2 실행 가드

2.2.1 역할에 따른 구조 불명확

기존 코드는 `parse_args()` 함수를 제외한 모든 함수가 `if __name__ == "__main__":` 실행 가드 내부에서 정의되고 동시에 사용된다. 이는 역할에 따른 구조 구분이 불명확하여 가독성을 떨어뜨리고, 향후 실험이 확장되었을 때 벤치마크 코드나 다른 모듈에서 개별 함수를 `import` 하여 사용할 수 없어 재사용성이 떨어진다.

2.3 성능 측정 및 비교 기능의 부재

2.3.1 실행 시간과 메모리 사용량을 정량적으로 측정하는 구조의 부재

기존 코드는 처리가 실패한 영상에 대해 영상, 프롬프트 조합을 기록할 뿐 각 실행에 대해 실행 시간, `peak memory` 등을 체계적으로 기록하는 기능이 없으며 이러한 정보는 실행 중에 표시되는 SAM3 자체 로그에 의존하고 있다. 따라서 실험 환경을 설정하거나, 실험을 개선하기 위해 필요한 정보를 파악하기 어렵다. 특히, 기존에 발생했던 특정 영상과 프롬프트 조합에서의 OOM 문제의 원인을 파악하기 위해서는 개별 예측마다 실행 시간과 메모리 사용량을 측정하고 저장하는 구조가 필요하다.

3. 적용 개념

본 개선에서는 기존 코드의 구조적 문제와 메모리 사용 문제를 해결하기 위해 수업에서 다룬 개념을 다음 세 가지를 중심으로 적용하였다.

3.1 Class 기반 구조 개선 및 단일책임원칙

기존 코드의 핵심 함수인 `run_video_with_prompts()`는 영상 프레임 로딩, SAM3 세션 관리, 프롬프트별 추론 실행, 결과 저장, 실패 처리, CUDA 메모리 정리, 진행 상황 출력을 모두 한 함수 안에서 처리했다. 때문에 특정 기능 수행을 위해서 함수 전체 흐름을 함께 이해해야 했으며, 추론 로직과 직접 얽혀 있는 세션 관리, 로깅 로직으로 인해 유지보수가 어려웠다.

이를 개선하기 위해 최종 코드에서는 실행 설정, 실행 단위, 실행 결과, 세션 관리, 출력 보고를 각각 별도의 객체와 함수로 분리했다. 목록은 다음과 같다.

- BatchRunConfig: 실행 중 변하지 않는 설정값
- VideoPromptJob: 하나의 영상-프롬프트 실행 단위
- PromptRunResult: 실행 결과 및 실패 정보
- RunReporter: 진행 상황 출력, 최종 실패 보고

이러한 역할 분리를 통해 추론 함수는 실제 프롬프트 실행에 집중하고, 설정 관리·상태 기록·출력 보고·자원 해제는 그 책임을 독립적으로 분리했다.

3.2 Iterator, Generator, Lazy Evaluation 기반 개선

기존 코드는 `read_video_frames_rgb()`에서 입력 영상의 모든 프레임을 리스트에 저장하고, SAM3의 `handle_stream_request()` 결과도 `outputs_per_frame` 딕셔너리에 모두 저장한 뒤, 두 자료구조를 한 번에 `save_masklet_video()`에 전달했다. 이 방식은 SAM3 코드의 처리 방식을 따른 것으로 구현은 단순하지만 전체 프레임과 예측 결과가 동시에 메모리에 유지된다. 특히 본 실험은 영상 데이터를 입력으로 사용하므로, 프레임수가 증가할수록 eager evaluation 방식에 의해 peak memory 사용량이 커지고, OOM 발생 가능성이 증가한다.

이러한 문제를 해결하고자 최종 코드에서는 다음과 같은 방법을 적용했다.

- `iter_propagated_outputs()`를 추가해 SAM3의 streaming 응답을 프레임 단위 generator로 전달
- `save_masklet_video_streaming()`을 구현하여 원본 영상을 `cv2.VideoCapture`로 한 프레임씩 읽고, SAM3 output stream에서 현재 프레임에 해당하는 예측 결과만 받아 렌더링한 뒤 `cv2.VideoWriter`에 즉시 기록

이를 통해 전체 프레임과 전체 예측 결과를 한 번에 저장하지 않고, "읽기 -> 현재 프레임 output 매칭 -> 렌더링 -> 저장"의 흐름으로 처리하도록 변경했다.

3.3 Decorator 기반 개선

기존 코드는 실패한 영상-프롬프트 조합을 출력할 뿐, 개별 실행에 소요된 시간이나 peak memory를 정량적으로 기록하지 않았다. 따라서 특정 조합에서 OOM이 발생하거나 실행 시간이 길어져도, 이를 비교 가능한 수치로 남기고 실험 개선의 근거로 삼기가 어려웠다. 따라서 최종 코드에서는 `measure_prompt_run` 데코레이터를 구현하여 `run_single_prompt_job()`에 적용하였다. 이 데코레이터는 핵심 추론 로직을 변경하지 않고, 실행 전후에 wall_clock time, Python-level peak memory, CUDA peak allocated memory를 측정한다. 이 때, `functools.wraps`를 사용해 원본 함수의 이름과 메타데이터가 유지되도록 하였다.

이를 통해 추론 함수 자체의 본질적인 책임이 아니라, 여러 함수에서 재사용될 수 있는 부가 기능인 측정과 기록을 분리하여, 추론 로직의 가독성을 유지하면서도 성능 측정을 자동화했다. 측정

결과는 3.1에서 언급한 PromptRunResult에 부착되고, 최종적으로 prompt_run_metrics.csv에 저장된다.

최종적으로 기존 코드는 다음과 같은 의도로 수정되었다.

- Class 기반 구조 개선 및 책임 분리로 유지보수성, 재사용성을 높인다.
- Generator 기반 streaming 처리로 영상 처리의 메모리 병목을 줄인다.
- Decorator 기반 실행 시간, 메모리 사용량 측정으로 핵심 추론 로직을 침범하지 않고 성능 분석 지표를 측정한다.

4. 개선 과정

코드 개선은 과제 설명에서 제시된 개선 항목 중 세 가지를 위의 3에서 언급한 순서대로 순차적으로 적용했다. 최종 수정본은 run_sam3_batch_after.py이며, 중간 수정본인 run_sam3_batch_afterC.py는 3.1 항목까지, run_sam3_batch_afterCB.py는 이어서 3.2 항목까지 반영한 수정본이다. 최종 수정본은 3.1, 3.2, 3.3 항목을 모두 반영했다.

4.1 Class 기반 구조 개선 및 단일책임원칙 적용

4.1.1 설정값과 실행 단위, 실행 결과 분리

```
100     def run_video_with_prompts(  
101         predictor,  
102         video_path: Path,  
103         prompts: List[str],  
104         out_root: Path,  
105         suffix: str,  
106         fps_out: Optional[float],  
107         skip_existing: bool,  
108         alpha: float = 0.5,  
109         frame_index_for_prompt: int = 0,  
110     ) -> List[str]:  
111  
112         failed_prompts = []  
113         # 프레임 로드  
114         video_frames_for_vis = read_video_frames_rgb(str(video_path))  
115  
116         if fps_out is None:  
117             fps_out = get_video_fps(str(video_path), default_fps=10.0)  
118     ...
```

기존 코드는 실행 설정값과 런타임 상태가 명확히 분리되어 있지 않았다. run_video_with_prompts()는 video_path, prompts, out_root, suffix, fps_out, skip_existing, alpha, frame_index_for_prompt 등을 개별 인자로 직접 전달받는 동시에 함수 내부에서 session_id, failed_prompts, outputs_per_frame 같은 실행 중 변화하는 상태도 함께 관리했다. 실행 과정에서 변하지 않는 설정값과 실행 중 계속 바뀌는 상태값이 한 함수 안에 섞여 있었기 때문에 함수의 인자 목록이 길어지고, 어떤 값이 실험 설정이고 어떤 값이 런타임 결과인지 코드만 보고 구분하기 어려웠다.

```

60 @dataclass(frozen=True)
61 class BatchRunConfig:
62     input_dir: Path
63     output_dir: Path
64     suffix: str
65     alpha: float
66     fps_out: Optional[float]
67     skip_existing: bool
68     prompt_frame: int
69     gpu_ids: Optional[str]
70
71 @dataclass(frozen=True)
72 class VideoPromptJob:
73     video_path: Path
74     prompt: str
75     output_path: Path
76
77 @dataclass
78 class PromptRunResult:
79     video_name: str
80     prompt: str
81     output_path: Path
82     status: str
83     error: Optional[str] = None
84     elapsed_sec: Optional[float] = None
85     peak_cpu_memory_mb: Optional[float] = None
86     peak_cuda_memory_mb: Optional[float] = None

```

이를 개선하기 위해 최종 수정본에서는 실행 설정, 실행 단위, 실행 결과를 각각 dataclass로 분리했다.

- **BatchRunConfig**는 실행 중 바뀌지 않는 설정값을 한 객체로 묶는다. input_dir, output_dir, suffix, alpha, fps_out, skip_existing, prompt_frame, gpu_ids는 실험 실행 전에 결정되는 값이므로 설정 객체에 포함했다. 또한 frozen=True를 사용하여 실행 도중 의도치 않게 설정값이 변경되지 않도록 했다.
- **VideoPromptJob**은 하나의 실행 단위를 나타낸다. 기존 코드에서는 영상과 프롬프트 조합을 중첩 반복문 안에서 즉석으로 처리했지만, 수정 후에는 “어떤 영상에 어떤 프롬프트를 적용하고 어느 경로에 저장할 것인가”를 하나의 job으로 명시했다. 이를 통해 전체 실행 대상의 개수, skip 대상, 실패 대상을 구조적으로 추적할 수 있게 되었다.
- **PromptRunResult**는 실행 결과를 나타낸다. 기존 코드에서는 실패한 프롬프트만 문자열 리스트로 반환했지만, 수정 후에는 각 job의 성공, 실패, skip 상태를 모두 객체로 저장한다. 최종 수정본에서는 여기에 실행 시간과 peak memory 측정값까지 포함하여, 이후 CSV 저장과 성능 비교에 사용할 수 있도록 했다.

이러한 변경은 기존 run_video_with_prompts()에 섞여 있던 설정값, 실행 단위, 실행 결과를 분리하여 코드의 의미를 명확히 한다. 그 결과 함수 인자가 간결해지고, 실행 결과가 단순 문자열이 아니라 구조화된 데이터로 남게 되어 후속 분석이 쉬워졌다.

4.1.2 SAM3 세션 관리 책임 분리

```

100 def run_video_with_prompts(
128     continue
129
130     if session_id is None:
131         # 세션 시작
132         response = predictor.handle_request(
133             request=dict(
134                 type="start_session",
135                 resource_path=str(video_path),
136             )
137         )
138         session_id = response["session_id"]
139     else:
140         try:
141             predictor.handle_request(
142                 request=dict(
143                     type="reset_session",
144                     session_id=session_id,
145                 )
146             )
147         except Exception:
148             # reset이 안 될 땐 세션 닫고 재시작
149             predictor.handle_request(
150                 request=dict(
151                     type="close_session",
152                     session_id=session_id,
153                 )
154             )

```

기존 코드에서는 SAM3 세션의 start, reset, close가 모두 run_video_with_prompts() 내부에 직접 명시되어 있었다.

```

100 def run_video_with_prompts(
160     )
161     session_id = response["session_id"]
162     cleanup_cuda()
163
164 > try: ...
195 except torch.OutOfMemoryError as e:
196     print(f"\n[OOM ERROR] {video_path.name} + '{prompt}'\n")
197     predictor.handle_request(request=dict(type="close_session", session_id=session_id))
198     session_id = None
199     failed_prompts.append(prompt)
200 except Exception as e:
201     print(f"\n[ERROR] {video_path.name} + '{prompt}'\n{str(e)}\n")
202     predictor.handle_request(request=dict(type="close_session", session_id=session_id))
203     session_id = None
204     failed_prompts.append(prompt)
205 finally:
206     if 'outputs_per_frame' in locals():
207         del outputs_per_frame
208     cleanup_cuda()
209
210 finally:
211     if session_id is not None:
212         # 세션 종료 + 메모리 정리
213         predictor.handle_request(
214             request=dict(
215                 type="close_session",
216                 session_id=session_id,
217             )
218         )
219     cleanup_cuda()
220
221 return failed_prompts

```

또한 함수 마지막의 finally 블록에서 세션 종료와 CUDA 정리를 직접 수행했다. 때문에 세션 및

메모리 관리 로직이 추론 흐름과 얽혀 있어 예외 처리 흐름이 복잡했고, 함수의 책임이 불명확했다. 이 때문에 세션 관리 정책을 바꾸기 위해서는 추론 함수의 전체 로직을 파악해야 해 유지보수가 어려웠다.

```
238 class Sam3VideoSession:
239     """
240     SAM3 predictor 세션의 start, reset, close 책임을 담당.
241     """
242
243     def __init__(self, predictor, video_path: Path):
244         self.predictor = predictor
245         self.video_path = video_path
246         self.session_id: Optional[str] = None
247
248     def __enter__(self) -> "Sam3VideoSession":
249         self.start()
250         return self
251
252     def __exit__(self, exc_type, exc, tb) -> bool:
253         self.close()
254         cleanup_cuda()
255
256         return False
257
258     def start(self) -> None:
259 >         response = self.predictor.handle_request(...)
260         self.session_id = response["session_id"]
261
262     def reset(self) -> None:
263 >         if self.session_id is None: ...
264
265         try:
266 >             self.predictor.handle_request(...)
267 >         except Exception: ...
268
269     def close(self) -> None:
270 >         if self.session_id is None: ...
271
272         try: ...
273         finally:
274             self.session_id = None
275
276     def require_session_id(self) -> str:
277 >         if self.session_id is None: ...
278         return self.session_id
279
280
```

이를 해결하기 위해, 수정본에서는 SAM3 세션의 생명주기 관리를 전담하는 **Sam3VideoSession** 클래스를 도입하여, 영상 단위 실행 함수에서 해당 클래스를 context manager로 사용한다.


```

636 def run_video_jobs(
637     predictor,
638     video_path: Path,
639     jobs: List[VideoPromptJob],
640     config: BatchRunConfig,
641 ) -> List[PromptRunResult]:
642     """
643     하나의 영상에 속한 여러 prompt job을 실행.
644     기존 run_video_with_prompts()가 하던 비디오 단위 처리를 대체함.
645     """
646
647     if not jobs:
648         return []
649
650     runnable_jobs, skipped_results = filter_runnable_jobs(
651         jobs=jobs,
652         config=config,
653     )
654
655     if not runnable_jobs:
656         return skipped_results
657
658     if config.fps_out is None:
659         resolved_fps_out = get_video_fps(video_path, default_fps=10.0)
660     else:
661         resolved_fps_out = config.fps_out
662
663     results: List[PromptRunResult] = []
664     results.extend(skipped_results)
665
666     with Sam3VideoSession(predictor, video_path) as session:
667         for job in runnable_jobs:
668             result = run_single_prompt_job(
669                 predictor=predictor,
670                 job=job,
671                 session=session,
672                 fps_out=resolved_fps_out,
673                 config=config,
674             )
675             results.append(result)
676
677     cleanup_cuda()
678
679     return results

```

이를 통해 세션의 start, reset, close 책임을 Sam3VideoSession으로 이동했다. 해당 클래스는 `__enter__()`에서 세션을 시작하고, `__exit__()`에서 세션을 닫고 CUDA 메모리를 정리한다. 따라서 예외가 발생하더라도 context manager의 종료 과정에서 자원 정리가 수행된다.

이러한 변경은 수업에서 다룬 context manager 개념을 적용한 것으로, 기존 코드의 try-finally 기반 자원 해제 코드를 객체 안에 캡슐화하여, 추론 함수가 세션 해제 책임을 직접 갖지 않도록 했다. 또한 `__exit__()`이 False를 반환하도록 하여 세션 해제 과정 이후에도 원래 예외가 숨겨지지 않도록 한다. 그 결과 `run_single_prompt_job()`과 `run_video_jobs()`는 세션을 사용해 job을 실행하는 것에 집중할 수 있게 되었다.

4.1.3 진행 상황 출력과 실패 보고 분리

수정 전 코드에서는 진행 상황 출력과 실패 보고가 추론 흐름 안에 직접 작성되어 있었다.

```

100 def run_video_with_prompts(
120
121     try:
122         for prompt in prompts:
123             pslug = slug_prompt(prompt)
124             out_path = out_root / f"{video_path.stem}{suffix}_{pslug}.mp4"
125
126             if skip_existing and out_path.exists():
127                 print(f"[SKIP] {video_path.name} + '{prompt}' -> {out_path.name} (already exists)")
128                 continue
129
130 >         if session_id is None: ...
139 >         else: ...
163
164 >         try: ...
195         except torch.OutOfMemoryError as e:
196             print(f"\n[OOM ERROR] {video_path.name} + '{prompt}'\n")
197             predictor.handle_request(request=dict(type="close_session", session_id=session_id))
198             session_id = None
199             failed_prompts.append(prompt)
200         except Exception as e:
201             print(f"\n[ERROR] {video_path.name} + '{prompt}'\n{str(e)}\n")
202             predictor.handle_request(request=dict(type="close_session", session_id=session_id))

```

또한 전체 실행이 끝난 뒤, main 흐름에서 직접 실패 조합을 출력했다.

```

262         for fp in failed_prompts:
263             failed_combis.append((vp.name, fp))
264
265         print("\n[DONE] ALL video files processed.")
266
267         if failed_combis:
268             print("\n" + "=" * 60)
269             print("[REPORT] FAILED COMBINATIONS]")
270             print("=" * 60)
271             for v_name, p in failed_combis:
272                 print(f"- Video: {v_name} + Prompt: '{p}'")
273             print("=" * 60 + "\n")
274         else:
275             print("[REPORT] No failures to report.")

```

따라서 추론 함수가 실제 예측뿐 아니라 사용자에게 어떤 메시지를 출력할지도 함께 결정하여 역할 분리가 잘 되지 않았으며, 출력 형식을 바꾸거나 실패 보고 방식을 바꾸려면 추론 함수 내부를 수정해야 했다.

```

303 class RunReporter:
304     """
305     진행 상황 출력과 최종 실패 보고를 담당.
306     """
307
308     def on_start(self, videos: List[Path], jobs: List[VideoPromptJob]) -> None:
309         print(f"[INFO] Found {len(videos)} video file(s).")
310         print(f"[INFO] Planned {len(jobs)} video-prompt job(s).")
311
312     def on_video_start(self, video_path: Path, num_jobs: int) -> None:
313         print(f"\n[VIDEO] Processing {video_path.name} ({num_jobs} prompt job(s))")
314
315     def on_result(self, result: PromptRunResult) -> None:
316         metric_text = ""
317
318         > if result.elapsed_sec is not None: ...
319
320         > if result.peak_cpu_memory_mb is not None: ...
321
322         > if result.peak_cuda_memory_mb is not None: ...
323
324         > if result.status == "done": ...
325         > elif result.status == "skipped": ...
326         else:
327             print(
328                 f"\n[ERROR] {result.video_name} + '{result.prompt}'{metric_text}\n"
329                 f"{result.error}\n"
330             )
331
332     def on_finish(self, results: List[PromptRunResult]) -> None:
333         print("\n[DONE] All video files processed.")
334
335         failed_results = [r for r in results if r.status == "failed"]
336
337         if not failed_results:
338             print("[REPORT] No failures to report.")
339             return
340
341         print("\n" + "=" * 60)
342         print("[REPORT] FAILED COMBINATIONS")
343         print("=" * 60)
344
345         for result in failed_results:

```

최종 수정본에서는 이러한 역할을 **RunReporter** 클래스로 분리했다. main 함수에서는 이 reporter의 메서드를 호출하는 방식으로 진행 상황을 출력한다.

```

681 def main() -> int:
707     reporter = RunReporter()
708     reporter.on_start(videos, jobs)
709
710     gpus_to_use = list(range(torch.cuda.device_count()))
711     print(f'[GPU] CUDA_VISIBLE_DEVICES={os.environ.get('CUDA_VISIBLE_DEVICES')}')
712     print(f'[GPU] using visible cuda device indices: {gpus_to_use}')
713
714     predictor = None
715     all_results: List[PromptRunResult] = []
716
717     try:
718         print("Loading SAM3 predictor...")
719         predictor = build_sam3_video_predictor(gpus_to_use=gpus_to_use)
720
721         for video_path in videos:
722             video_jobs = jobs_by_video.get(video_path, [])
723             reporter.on_video_start(video_path, len(video_jobs))
724
725             video_results = run_video_jobs(
726                 predictor=predictor,
727                 video_path=video_path,
728                 jobs=video_jobs,
729                 config=config,
730             )
731
732             for result in video_results:
733                 reporter.on_result(result)
734
735             all_results.extend(video_results)
736
737         finally:
738             if predictor is not None:
739                 try:
740                     predictor.shutdown()
741                 except Exception:
742                     pass
743
744             cleanup_cuda()
745
746     reporter.on_finish(all_results)

```

이러한 변경의 이유는 진행 상황 출력과 실패 보고가 추론 자체의 핵심 책임이 아니기 때문이다. 추론 함수는 job을 실행하고 결과 객체를 반환하면 되고, 그 결과를 콘솔에 어떻게 보여줄지는 별도 객체가 담당한다. 이러한 명확한 역할 분담은 함수의 재사용성과 유지보수성을 높인다. 일례로, 향후 로그 출력의 형식을 바꾸거나, 콘솔 출력이 아니라 로그 파일, JSON, CSV 등으로 변경하고 싶을 때도 추론 로직을 살필 필요 없이 RunReporter 클래스만으로 내용을 파악해 기능을 확장할 수 있다.

또한 최종 수정본에서는 decorator가 측정한 실행 시간과 peak memory도 PromptRunResult에 저장되므로, RunReporter는 같은 결과 객체를 기반으로 성능 지표까지 함께 출력할 수 있다.

4.1.4 실행 가드 내부 함수 정의를 모듈 수준으로 이동

```

48
49 if __name__ == "__main__":
50     # 환경 변수 세팅
51     args = parse_args()
52
53     if args.gpu_ids is not None:
54         os.environ["CUDA_VISIBLE_DEVICES"] = args.gpu_ids
55         print(f'[GPU] using CUDA_VISIBLE_DEVICES={args.gpu_ids}')
56
57     # Helper
58 > def slug_prompt(prompt: str, max_len: int = 40) -> str: ...
63
64
65 > def list_videos(folder: str): ...
72
73 > def read_video_frames_rgb(video_path: str): ...
87
88 > def get_video_fps(video_path: str, default_fps: float = 10.0) -> float: ...
95
96 > def cleanup_cuda(): ...
99
100 > def run_video_with_prompts(...) ...
101

```

수정 전 코드에서는 `parse_args()`를 제외한 모든 함수가 `if __name__ == "__main__"` 실행 가드 내부에 정의되어 있었다. 이 방식은 스크립트를 단독 실행할 때는 동작하지만, 다른 모듈이나 벤치마크 코드에서 개별 함수를 import하여 재사용하기 어렵다. 또한 파일을 읽을 때 실행 흐름과 함수 정의가 한 블록에 섞여 있어 구조 파악이 어렵다.

```

303 > class RunReporter: ...
361
362 > def save_results_csv(results: List[PromptRunResult], csv_path: Path) -> None: ...
397
398 > def iter_propagated_outputs(predictor, session_id: str): ...
410
411
412 > def measure_prompt_run(func): ...
462
463
464 > def save_masklet_video_streaming(...) ...
579
580 @measure_prompt_run
581 > def run_single_prompt_job(...) ...
634
635
636 > def run_video_jobs(...) ...
680
681 > def main() -> int: ...
753
754
755 if __name__ == "__main__":
756     sys.exit(main())

```

최종 수정본에서는 helper 함수, dataclass, session class, reporter class, 실행 함수가 모두 모듈 수준에 정의되고, 실행 가드에서는 `main()`만 호출한다. 이를 통해 모듈 단위 재사용이 가능해져, 향후 구조가 확장되었을 때 다른 벤치마크 코드에서 본 코드에 있는 내부 함수, 객체들을 import하여 사용할 수 있다.

해당 변경은 코드 구조를 “정의부”와 “실행부”로 나눈다는 점에서 중요하다. 기존 구조에서는 스크립트 실행과 동시에만 함수가 의미를 가졌지만, 수정 후에는 각 함수가 독립적인 모듈 구성 요소가 되었다. 따라서 향후 benchmark 코드, 테스트 코드, 다른 실험 스크립트에서 같은 로직을

재사용할 수 있다.

4.2 Iterator, Generator, Lazy Evaluation 기반 개선

4.2.1 전체 프레임 리스트, 딕셔너리 저장 제거

수정 전 코드에서는 메인 추론 함수 내부에서 `read_video_frames_rgb()`를 통해 영상을 처리하기 전에 먼저 모든 프레임을 읽어 `video_frames_for_vis` 리스트로 저장하였다.

```
73 def read_video_frames_rgb(video_path: str):
74     """mp4 비디오 -> 프레임 리스트(RGB)"""
75     cap = cv2.VideoCapture(video_path)
76     if not cap.isOpened():
77         raise RuntimeError(f'Cannot open video: {video_path}')
78
79     video_frames_for_vis = []
80     while True:
81         ret, frame = cap.read()
82         if not ret:
83             break
84         video_frames_for_vis.append(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
85     cap.release()
86     return video_frames_for_vis
```

뿐만 아니라, SAM3의 `handle_stream_request()`가 반환하는 frame별 예측 결과를 모두 `outputs_per_frame` 딕셔너리에 저장하였다.

```
100 def run_video_with_prompts(
172     )
173 )
174 # 비디오로 예측 전파하고 수집
175 outputs_per_frame = {}
176 for r in predictor.handle_stream_request(
177     request=dict(
178         type="propagate_in_video",
179         session_id=session_id,
180         propagation_direction="forward",
181     )
182 ):
183     outputs_per_frame[r["frame_index"]] = r["outputs"]
```

이후 전체 프레임 리스트와 전체 예측 결과 딕셔너리를 함께 `save_masklet_video`에 전달했다.

```
100 def run_video_with_prompts(
183     outputs_per_frame[r["frame_index"]] = r["outputs"]
184
185     # 비디오 저장
186     save_masklet_video(
187         video_frames_for_vis,
188         outputs_per_frame,
189         out_path=str(out_path),
190         fps=fps_out,
191         alpha=alpha,
192     )
193
194     print(f'[DONE] {video_path.name} + '{prompt}' -> {out_path.name}')
```

따라서 우선적으로, 매 실행마다 입력 프레임 전체를 한 번에 메모리에 저장해, 영상 하나를 메모리에 올리는 꼴이 되어 입력 영상의 길이가 길어지거나, 해상도가 높아졌을 때 메모리 사용량이 증가했다. 또한, 이 구조에서는 입력 프레임 전체와 예측 결과 전체가 동시에 메모리에 존재하기 때문에, 따라서 SAM3 추론 결과가 큰 경우, 또는 프레임 수가 많은 경우 메모리 사용량이 크게 증가한다.

이러한 구조는 SAM3의 공개 코드가 제공하는 `save_masklet_video()` 함수가 전체 영상 프레임과 예측 결과를 리스트와 딕셔너리로 받기 때문에 발생한 문제로, 이를 해결하기 위해 다음과 같은 함수들을 추가했다.

```
464 def save_masklet_video_streaming(
465     video_path: Path,
466     output_stream,
467     out_path: Path,
468     alpha: float = 0.5,
469     fps: float = 10,
470 ) -> None:
471     """
472     기존 SAM3 save_masklet_video()와 유사하게 동작하되,
473     전체 frame list와 전체 outputs dict를 만들지 않는 streaming 저장 함수.
474     """
475     cap = cv2.VideoCapture(str(video_path))
476
477 def save_masklet_video_streaming(
478     if not cap.isOpened(): ...
479
480     temp_file = NamedTemporaryFile(...)
481     temp_path = temp_file.name
482     temp_file.close()
483
484     writer = None
485     progress = None
486
487     try:
488         width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
489         height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
490
491         if width <= 0 or height <= 0: ...
492
493         fourcc = cv2.VideoWriter_fourcc(*"mp4v")
494         writer = cv2.VideoWriter(temp_path, fourcc, fps, (width, height))
495
496         if not writer.isOpened(): ...
497
498         output_iter = iter(output_stream)
499
500         try:
501             next_frame_idx, next_outputs = next(output_iter)
502         except StopIteration: ...
503
504         frame_idx = 0
505         progress = tqdm(desc=f"Saving {out_path.name}", unit="frame")
506
507         while True:
508             ret, frame_bgr = cap.read()
509             if not ret:
510                 break
511
512             frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
513             img = load_frame(frame_rgb)
```

```

def save_masklet_video_streaming(
    frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
    img = load_frame(frame_rgb)

    frame_outputs = None

    # output stream이 현재 frame보다 뒤쳐져 있으면 따라잡기.
    while next_frame_idx is not None and next_frame_idx < frame_idx:
        try:
            next_frame_idx, next_outputs = next(output_iter)
        except StopIteration:
            next_frame_idx, next_outputs = None, None
            break

    # 현재 frame에 해당하는 SAM3 output이 있으면 기존 SAM3 렌더링 함수를 사용.
    if next_frame_idx == frame_idx:
        frame_outputs = next_outputs

        try:
            next_frame_idx, next_outputs = next(output_iter)
        except StopIteration:
            next_frame_idx, next_outputs = None, None

    if frame_outputs is not None:
        overlay = render_masklet_frame(
            img,
            frame_outputs,
            frame_idx=frame_idx,
            alpha=alpha,
        )
    else:
        # output이 없는 frame은 원본 frame을 그대로 저장.
        overlay = img

    writer.write(cv2.cvtColor(overlay, cv2.COLOR_RGB2BGR))

    frame_idx += 1
    progress.update(1)

```

기존의 SAM3에서 제공하는 `save_masklet_video()` 함수는 전체 frame list와 전체 output dict를 입력으로 받는 방식이었기 때문에, 해당 코드를 유지하는 상황에서는 앞서 제시한 기존 코드의 처리 방식을 변경할 수 없었다. 따라서 기존 SAM3 시각화 로직을 참고하여, 전체 프레임과 예측 결과를 순차적으로 소비하는 새로운 시각화 함수 **`save_masklet_video_streaming()`**를 추가했다. 이를 통해 전체 프레임 리스트를 만드는 `read_video_frames_rgb()` 대신, 해당 함수 안에서 `cv2.VideoCapture`를 통해 프레임을 하나씩 읽는다. 이후, 현재 `frame_idx`와 SAM3 output의 `next_frame_idx`가 일치하면 mask overlay를 렌더링하고, 해당 프레임에 output이 없으면 원본 프레임을 그대로 저장한다. 이러한 방식은 전체 영상을 메모리에 올리지 않고도 결과 영상을 생성할 수 있게 한다. 또한 기존 SAM3 시각화 유틸리티의 `load_frame()`과 `render_masklet_frame()`을 재사용하여 mask_overlay 방식은 유지하였다.

```

398 def iter_propagated_outputs(predictor, session_id: str):
399     """
400     SAM3 predictor의 handle_stream_request() 결과를 frame 단위로 반환.
401     """
402     for response in predictor.handle_stream_request(
403         request=dict(
404             type="propagate_in_video",
405             session_id=session_id,
406             propagation_direction="forward",
407         )
408     ):
409         yield response["frame_index"], response["outputs"]

```


다음으로 SAM3의 streaming 응답을 generator로 감싼 `iter_propagated_outputs()`를 추가했다. `run_single_prompt_job()`에서는 이 generator를 생성한 뒤, `save_masklet_video_streaming()`에 전달한다.

```
581 def run_single_prompt_job(  
582     ,  
606  
607     output_stream = iter_propagated_outputs(  
608         predictor=predictor,  
609         session_id=session_id,  
610     )  
611  
612     save_masklet_video_streaming(  
613         video_path=job.video_path,  
614         output_stream=output_stream,  
615         out_path=job.output_path,  
616         alpha=config.alpha,  
617         fps=fps_out,  
618     )
```

기존에는 결과 시각화를 위해 전체 프레임과 예측 결과를 보관했지만, 수정 후에는 현재 처리 중인 프레임만 메모리에 유지한다. 결과 영상 저장 과정에서는 이전 프레임 전체가 필요하지 않고, 현재 프레임과 현재 프레임에 대응하는 segmentation output만이 요구된다. 따라서 “한 번만 순차적으로 사용되는 데이터는 전체를 저장하지 않는다”는 철학에 따라 영상 처리 흐름을 기존의 eager evaluation에서 lazy evaluation 방식으로 바꾸는 것이 적절하다. 이러한 변경은 peak memory를 줄이는 데 직접적으로 연결된다.

4.3 Decorator 기반 개선

4.3.1 단일 프롬프트 실행 함수에 성능 측정 decorator 적용

```

def run_video_with_prompts(
    ) -> List[str]:

    failed_prompts = []
    # 프레임 로드
    video_frames_for_vis = read_video_frames_rgb(str(video_path))

    if fps_out is None: ...

    session_id = None

    try:
        for prompt in prompts:
            pslug = slug_prompt(prompt)
            out_path = out_root / f"{video_path.stem}{suffix}_{pslug}.mp4"

            if skip_existing and out_path.exists(): ...

            if session_id is None: ...
            else: ...

            try: ...
            except torch.OutOfMemoryError as e: ...
            except Exception as e:
                print(f"\n[ERROR] {video_path.name} + '{prompt}'\n{str(e)}\n")
                predictor.handle_request(request=dict(type="close_session", session_id=session_id))
                session_id = None
                failed_prompts.append(prompt)
            finally:
                if 'outputs_per_frame' in locals():
                    del outputs_per_frame
                cleanup_cuda()

    finally:
        if session_id is not None:
            # 세션 종료 + 메모리 정리
            predictor.handle_request(
                request=dict(
                    type="close_session",
                    session_id=session_id,
                )
            )
        cleanup_cuda()

    return failed_prompts
print("\n[DONE] ALL video files processed.")

if failed_combis:
    print("\n" + "=" * 60)
    print("[REPORT] FAILED COMBINATIONS")
    print("=" * 60)
    for v_name, p in failed_combis:
        print(f"- Video: {v_name} + Prompt: '{p}'")
    print("=" * 60 + "\n")
else:
    print("[REPORT] No failures to report.")
finally:

```

수정 전 코드에서는 실행이 실패한 영상-프롬프트 조합을 저장한 후, 모든 실행 종료 후에 콘솔에 출력할 뿐, 실행 시간, CPU peak memory, CUDA peak memory 등을 별도로 측정하거나 저장하지 않았다. 따라서 실제 영상-프롬프트 실행 단위별로 걸리는 실행 시간이나 메모리 사용량을 파악할 수 없었고, OOM의 원인을 분석하거나 문제를 개선하기 위한 근거 정보를 획득하기 어려웠다. 이러한 문제를 해결하기 위해 수정본에서는 **measure_prompt_run** 데코레이터를 구현하고, 이를 단일 job 실행 함수인 `run_single_prompt_job()`에 적용하였다.

```

412 def measure_prompt_run(func):
413     """
414     단일 prompt job 실행 시간과 메모리 사용량을 측정하는 decorator.
415
416     측정 대상:
417     - elapsed_sec: wall-clock 실행 시간
418     - peak_cpu_memory_mb: tracemalloc 기준 Python-level peak memory
419     - peak_cuda_memory_mb: torch.cuda 기준 GPU peak allocated memory
420     """
421
422     @wraps(func)
423     def wrapper(*args, **kwargs):
424         was_tracing = tracemalloc.is_tracing()
425
426         if not was_tracing:
427             tracemalloc.start()
428
429         if torch.cuda.is_available():
430             torch.cuda.reset_peak_memory_stats()
431
432         start_time = time.perf_counter()
433
434         try:
435             result = func(*args, **kwargs)
436             return result
437
438         finally:
439             elapsed_sec = time.perf_counter() - start_time
440
441             current_bytes, peak_bytes = tracemalloc.get_traced_memory()
442             peak_cpu_memory_mb = peak_bytes / (1024 * 1024)
443
444             if not was_tracing:
445                 tracemalloc.stop()
446
447             if torch.cuda.is_available():
448                 peak_cuda_memory_mb = torch.cuda.max_memory_allocated() / (1024 * 1024)
449             else:
450                 peak_cuda_memory_mb = None
451
452             # func가 정상적으로 PromptRunResult를 반환한 경우에만 metric을 붙입니다.
453             # try-finally 구조상 return 직전 result를 직접 수정하기 위해 locals()를 사용합니다.
454             if "result" in locals() and isinstance(result, PromptRunResult):
455                 result.attach_metrics(
456                     elapsed_sec=elapsed_sec,
457                     peak_cpu_memory_mb=peak_cpu_memory_mb,
458                     peak_cuda_memory_mb=peak_cuda_memory_mb,
459                 )
460
461         return wrapper

```

```

580 @measure_prompt_run
581 def run_single_prompt_job(
582     predictor,
583     job: VideoPromptJob,
584     session: Sam3VideoSession,
585     fps_out: float,
586     config: BatchRunConfig,
587 ) -> PromptRunResult:
588     """
589     하나의 video-prompt job을 실행.
590     단일 프롬프트에 대한 add_prompt, propagate, save만 담당함.
591     """

```

이러한 구현 방식을 통해, 성능 측정 로직을 run_single_prompt_job() 내부에 직접 작성하지 않고 decorator로 분리한다. 따라서 run_single_prompt_job()은 "하나의 video-prompt job을 실행하고 결과를 반환하는 책임"만을 갖도록 유지하고, 실행 시간과 메모리 측정은 wrapper가 함수 호출 전후에 수행하도록 역할을 구분한다. 실행 시간과 peak memory 측정은 여러 함수에 재사용될 수 있는 부가 기능이며, 이를 decorator로 분리하면 핵심 로직을 오염시키지 않고 관찰 기능을 추가할 수 있다.

또한 @wraps(func)를 사용하여 원본 함수의 이름, docstring 등 메타데이터를 보존했다. 이는 데코레이터 사용 시 원본 함수의 의미와 디버깅 정보를 해치지 않기 위한 조치다.

4.3.2 측정 결과를 실행 결과 객체에 부착

```
121     try:
122         for prompt in prompts:
123             pslug = slug_prompt(prompt)
124             out_path = out_root / f'{video_path.stem}{suffix}_{pslug}.mp4'
125
126 >             if skip_existing and out_path.exists(): ...
129
130 >             if session_id is None: ...
139 >             else: ...
163
164 >             try: ...
195 >             except torch.OutOfMemoryError as e: ...
200             except Exception as e:
201                 print(f'\n[ERROR] {video_path.name} + '{prompt}'\n{str(e)}\n')
202                 predictor.handle_request(request=dict(type="close_session", session_id=session_id))
203                 session_id = None
204                 failed_prompts.append(prompt)
205             finally:
267         if failed_combis:
268             print("\n" + "="*60)
269             print("[REPORT] FAILED COMBINATIONS")
270             print("="*60)
271             for v_name, p in failed_combis:
272                 print(f'- Video: {v_name} + Prompt: '{p}''')
273             print("="*60 + "\n")
274         else:
275             print("[REPORT] No failures to report.")
276     ...
```

기존 코드에서 실행 결과는 실패한 프롬프트 이름 목록 중심으로 관리되었다. 따라서 성공한 프롬프트에 대해서는 별도의 구조화된 결과가 남지 않았고, 실패한 경우에도 영상명과 프롬프트 조합은 main 함수에서 다시 조립하였다. 이러한 방식은 성공, 실패, skip 결과를 동일한 형식으로 다루기 어렵고, 성능 측정값을 연결할 공간도 없다.

```

77 @dataclass
78 class PromptRunResult:
79     video_name: str
80     prompt: str
81     output_path: Path
82     status: str
83     error: Optional[str] = None
84     elapsed_sec: Optional[float] = None
85     peak_cpu_memory_mb: Optional[float] = None
86     peak_cuda_memory_mb: Optional[float] = None
87
88     @classmethod
89     > def done(cls, job: VideoPromptJob) -> "PromptRunResult": ...
96
97     @classmethod
98     > def skipped(cls, job: VideoPromptJob) -> "PromptRunResult": ...
105
106     @classmethod
107     > def failed(cls, job: VideoPromptJob, error: Exception) -> "PromptRunResult": ...
115
116     def attach_metrics(
117         self,
118         elapsed_sec: float,
119         peak_cpu_memory_mb: Optional[float],
120         peak_cuda_memory_mb: Optional[float],
121     ) -> "PromptRunResult":
122         self.elapsed_sec = elapsed_sec
123         self.peak_cpu_memory_mb = peak_cpu_memory_mb
124         self.peak_cuda_memory_mb = peak_cuda_memory_mb
125         return self

```

이를 해결하기 위해, 수정본에서는 PromptRunResult에 측정값 필드를 추가하고, attach_metrics() 메서드로 측정 결과를 부착한다. 그 결과 실행 결과 객체에 상태 정보와 측정 정보를 함께 저장한다. 이를 통해 하나의 PromptRunResult가 다음 정보를 모두 포함할 수 있게 되었다.

- 어떤 영상에 대한 결과인지
- 어떤 프롬프트에 대한 결과인지
- 출력 파일 경로가 무엇인지
- 실행 상태가 done, skipped, failed 중 무엇인지
- 실패했다면 에러 메시지가 무엇인지
- 실행 시간이 얼마였는지
- CPU peak memory가 얼마였는지
- CUDA peak memory가 얼마였는지

이 변경은 이후 결과를 콘솔에 출력하거나 CSV로 저장할 때 동일한 데이터 구조를 사용할 수 있게 한다. 또한 성능 측정값이 실행 결과와 분리되지 않고 같은 객체에 부착되므로, 영상-프롬프트 조합별 비교가 쉬워진다.

4.3.3 실행 결과 CSV 저장

앞선 4.3.1, 4.3.2에서 언급한 바와 같이, 기존 코드는 모든 실행이 끝난 뒤 실패한 영상-프롬프트 조합의 이름만을 콘솔에 출력하였다. 이는 실행 성공 실패 여부를 즉시 눈으로 확인하는 데는 충분하지만, 향후 실험 설계 개선 등을 위한 지표를 기록하고 정리하는 방식으로는 부적절하다. 따

라서 최종 수정본에서는 `save_results_csv()` 함수를 추가하여, 모든 실행 결과를 csv 파일로 저장한다.

```
362 def save_results_csv(results: List[PromptRunResult], csv_path: Path) -> None:
363     """
364     PromptRunResult 목록을 CSV로 저장.
365     보고서의 실행 시간 / 메모리 사용량 표 작성에 사용.
366     """
367     csv_path.parent.mkdir(parents=True, exist_ok=True)
368
369     fieldnames = [
370         "video_name",
371         "prompt",
372         "output_path",
373         "status",
374         "elapsed_sec",
375         "peak_cpu_memory_mb",
376         "peak_cuda_memory_mb",
377         "error",
378     ]
379
380     with csv_path.open("w", newline="", encoding="utf-8") as f:
381         writer = csv.DictWriter(f, fieldnames=fieldnames)
382         writer.writeheader()
383
384         for result in results:
385             writer.writerow(
386                 {
387                     "video_name": result.video_name,
388                     "prompt": result.prompt,
389                     "output_path": str(result.output_path),
390                     "status": result.status,
391                     "elapsed_sec": result.elapsed_sec,
392                     "peak_cpu_memory_mb": result.peak_cpu_memory_mb,
393                     "peak_cuda_memory_mb": result.peak_cuda_memory_mb,
394                     "error": result.error,
395                 }
396             )
397
681 def main() -> int:
733     reporter.on_result(result)
734
735     all_results.extend(video_results)
736
737     finally:
738         if predictor is not None:
739             try:
740                 predictor.shutdown()
741             except Exception:
742                 pass
743
744     cleanup_cuda()
745
746     reporter.on_finish(all_results)
747
748     result_csv_path = config.output_dir / "prompt_run_metrics.csv"
749     save_results_csv(all_results, result_csv_path)
750     print(f"[REPORT] Benchmark results saved to: {result_csv_path}")
751
752     return 0
```

이를 통해 성공, 실패, skip을 포함한 모든 job 결과가 CSV로 저장되고, 이후 실행 시간, peak memory, 실패 여부를 비교하는 근거 자료로 사용할 수 있다.

4.4 수정사항 종합 요약

최종 수정본은 기존의 `run_video_with_prompts()` 중심 구조를 해체하고, 실행 설정, 실행 단위, 실행 결과, 세션 관리, 출력 보고, 성능 측정, streaming 저장을 각각 분리했다.

4.1에서는 `class`와 `dataclass`를 사용하여 코드 구조를 정리했다.

- `BatchRunConfig`, `VideoPromptJob`, `PromptRunResult`로 설정, 실행 단위, 실행 결과 분리
- `Sam3VideoSession`으로 SAM3 세션의 생명주기를 context manager로 관리
- `RunReporter`로 진행 상황 출력과 실패 보고 수행
- 함수들을 실행 가드 밖으로 이동하고 `main()`을 도입하여 모듈 재사용성을 높임

4.2에서는 기존의 eager evaluation 구조를 streaming 구조로 바꿨다.

- 전체 프레임을 리스트로 저장하던 방식과 전체 예측 결과를 딕셔너리로 저장하던 방식을 제거
- `iter_propagated_outputs()`와 `save_masklet_video_streaming()`을 통해 프레임과 예측 결과를 순차적으로 소비
- 이를 통해 영상 길이와 해상도 증가에 따른 peak memory 증가를 줄이고자 했다.

4.3에서는 decorator를 활용하여 성능 측정 기능을 핵심 추론 함수와 분리했다.

- `measure_prompt_run`로 `run_single_prompt_job()` 실행 전후에 실행 시간, CPU peak memory, CUDA peak memory를 측정
- 측정 결과를 `PromptRunResult`에 부착
- `save_results_csv()`를 통해 모든 실행 결과를 `prompt_run_metrics.csv`로 저장

이를 통해 코드 성능의 정량적 비교가 가능해진다.

전체적으로, 최종 수정본은 단순히 코드 일부를 빠르게 만드는 방향이 아니라, 연구 코드의 실행 구조를 “설정 생성 → job 생성 → 영상별 job 그룹화 → session context에서 job 실행 → result 수집 → reporter 출력 → CSV 저장” 흐름으로 재구성한 것이다. 이 구조는 향후 benchmark 코드와 연결하기 쉽고, 추가 실험 조건이 생겨도 개별 구성 요소를 수정하는 방식으로 확장할 수 있다.

4.5 최적화 과정에서 발생한 trade-off

4.5.1 코드 길이와 구성 요소 수의 증가

최종 수정본은 `BatchRunConfig`, `VideoPromptJob`, `PromptRunResult`, `Sam3VideoSession`, `RunReporter`, `measure_prompt_run`, `save_masklet_video_streaming()` 등이 추가되면서 전체 코드량이 증가했다. 대신 각 구성 요소의 책임이 명확해졌고, 이를 통해 특정 기능을 수정하기 위한 변경 범위가 좁아졌다.

4.5.2 streaming 저장 방식 적용으로 인한 구현 복잡도

기존 `save_masklet_video()`는 전체 프레임 리스트와 전체 output 디렉토리를 넘기면 되므로 사용하기 쉬웠다. 반면 `save_masklet_video_streaming()`은 원본 영상 프레임 번호와 SAM3 output stream의 frame index를 맞추는 로직이 필요하고, 임시 파일 생성과 ffmpeg 재인코딩도 직접 관리해야 한다. 따라서 메모리 효율을 얻는 대신 저장 함수의 복잡도는 증가했다.

4.5.3 decorator 기반 성능 측정으로 인한 약간의 실행 오버헤드

`time.perf_counter()`, `tracemalloc`, `torch.cuda.reset_peak_memory_stats()`, `torch.cuda.max_memory_allocated()`를 매 job마다 호출하므로 측정 자체의 비용이 발생한다.

5. 최적화 결과

실험 환경은 다음과 같다.

- OS: Ubuntu 22.04.3 LTS
- Python 버전: Python 3.12
- Torch 버전: CUDA 12.6 wheel, PyTorch 2.7.0
- 반복횟수: 3회
- 입력: pixabay에서 수집하고 편집한 HD, 10fps, 15초 이내의 브로콜리 등장 요리 영상 4개

benchmark 함수에서 측정하는 지표는 각각 다음과 같다.

- **elapsed_sec**: benchmark 대상 스크립트를 실행한 직후부터 해당 프로세스가 종료될 때까지의 전체 wall-clock 실행 시간이다. Python의 `time.perf_counter()`를 사용해 측정하며, 모델 로딩, 입력 처리, 추론, 결과 저장, 후처리, 하위 프로세스 실행 시간이 모두 포함된다.
- **rss_peak**: 벤치마크 대상 프로세스와 하위 프로세스들이 실제 물리 메모리에 적재한 최대 RSS 메모리 사용량이다. 공유 메모리가 포함될 수 있어 실제 고유 사용량보다 크게 측정될 수 있다.
- **uss_peak**: 벤치마크 대상 프로세스와 하위 프로세스들이 고유하게 점유한 최대 USS 메모리 사용량이다. 공유 메모리 영향을 줄인 값이므로 개별 실험의 CPU 메모리 사용량 비교에 더 적합하다.
- **system_memory_peak**: 실험 수행 중 시스템 전체에서 사용된 CPU 메모리의 최대값이다. 다른 실험이나 백그라운드 프로세스의 메모리 사용량도 포함되므로 개별 실험의 메모리 사용량으로 해석하지 않는다.
- **system_memory_percent_peak**: 실험 수행 중 시스템 전체 메모리 사용률의 최대값이다. 전체 서버 또는 로컬 환경의 메모리 압박 정도를 확인하기 위한 참고 지표이다.
- **gpu_peak**: `nvidia-smi`를 통해 벤치마크 대상 프로세스 트리의 GPU 메모리 사용량을 추적한 최대값이다. 단일 GPU 실행에서는 해당 GPU의 최대 사용량으로, 멀티 GPU 실행에서는 사용된 GPU들의 메모리 사용량 합산 peak로 해석한다.

본 개선은 SAM3 모델 자체의 내부 연산을 변경하는 것이 아니라, 기존 실험 코드의 실행 구조, 중간 결과 저장 방식, 성능 측정 방식을 개선하는 것을 목표로 한다. 따라서 최적화 결과는 단순히 실행 시간이 얼마나 감소했는지만으로 판단하기보다, 다음과 같은 관점에서 비교할 필요가 있다.

- 실행 시간 비교
- 메모리 사용량 비교
- 입력 크기별 성능 변화
- 코드 구조 변화
- 중복 제거 결과
- 재사용성 또는 확장성 개선 결과
- 실험 조건 변경 시 코드 수정 범위 비교

5.1 실행 시간 비교

실행 시간 측면에서, 본 과제에서 수행한 개선은 기존 코드에 비해 SAM3 추론 자체를 크게 빠르게 만드는 구조는 아니다. 전체 실행 시간의 상당 부분은 `predictor.handle_request()`, `predictor.handle_stream_request()`와 같은 SAM3 내부 추론 과정에서 발생한다. 해당 부분은 입력 영상의 길이, 해상도, 프롬프트, GPU 상태, SAM3 내부 구현에 크게 의존하므로, Python 코드의 구조를 바꾼다고 해서 모델 추론 시간이 직접적으로 크게 줄어들지 않는다.

다만 수정본에서는 실행 단위를 `VideoPromptJob`으로 명시하고, `filter_runnable_jobs()`를 통해 실행할 job과 skip할 job을 분리했다. 기존 코드에서는 영상의 프레임을 먼저 읽은 뒤 프롬프트별 출력 파일 존재 여부를 검사하는 구조였기 때문에, 이미 결과가 존재하여 실제 추론이 필요 없는 경우에도 불필요한 프레임 로딩이 발생할 수 있었다. 수정 후에는 먼저 실행 대상 job을 분리하기 때문에, 이미 처리된 영상-프롬프트 조합이 많은 재실행 상황에서는 불필요한 실행 과정을 줄일 수 있다.

실행 시간 비교 결과는 다음과 같다. 단위는 초다.

	수정 전	수정 후	차이
평균 시간	1576.34 ± 32.36	1538.78 ± 48.23	약 37.56초, 2.38% 감소
최소	1539.14	1506.64	
최대	1597.96	1594.24	

실행 시간은 평균적으로 약 37초 감소했으나, 전체 실행 시간이 약 1500초 이상으로 매우 길기 때문에 개선 폭은 2.38% 수준에 그쳤다. 이는 전체 실행 시간의 대부분이 SAM3 내부 추론 과정에서 발생하기 때문으로 보인다. 따라서 본 개선은 모델 추론 자체를 크게 단축했다기보다, 실행 구조 정리, 불필요한 중간 처리 감소를 통해 소폭의 실행 시간 개선을 보인 것으로 판단할 수 있다.

5.2 메모리 사용량 비교

기존 코드의 메모리 사용 문제는 크게 두 가지에서 발생했다. 첫째, `read_video_frames_rgb()`가 입력 영상의 전체 프레임을 리스트로 저장했다. 둘째, `handle_stream_request()`로부터 반환되는 전체 예측 결과를 `outputs_per_frame` 딕셔너리에 저장했다. 이후 전체 프레임 리스트와 전체 예측 결과 딕셔너리를 `save_masklet_video()`에 전달했기 때문에, 영상 프레임과 예측 결과가 동시에 메모리에 유지되었다.

최종 수정본에서는 이러한 eager evaluation 구조를 줄이기 위해 `iter_propagated_outputs()`와 `save_masklet_video_streaming()`을 추가했다. 이를 통해 SAM3의 streaming 응답을 generator 형태로 소비하고, 원본 영상 역시 `cv2.VideoCapture`를 통해 한 프레임씩 읽도록 변경했다. 따라서 Python 코드 수준에서는 전체 프레임 리스트와 전체 예측 결과 딕셔너리를 동시에 보관하지 않게 되었다.

이론적으로 이 변경은 Python-level peak memory를 줄이는 데 도움이 된다. 특히 영상 길이가 길거나 해상도가 높은 경우, 전체 프레임을 리스트로 저장하지 않는 효과가 커질 수 있다. 또한 SAM3 output이 많은 경우에도 전체 결과를 딕셔너리에 저장하지 않으므로, Python 객체로 유지되는 중간 결과의 양을 줄일 수 있다.

다만 SAM3는 영상 segmentation과 tracking을 수행하기 위해 내부적으로 feature, session state, propagation 과정의 tensor 등을 유지하기 때문에, Python 코드에서 frame list나 `outputs_per_frame` 딕셔너리를 제거하더라도 이러한 메모리 사용량이 그대로 남을 수 있다.

메모리 사용량 비교 결과는 다음과 같다. 단위는 MB다.

	수정 전	수정 후	차이
평균 peak RSS	7673.52 ± 7.17	7616.28 ± 92.34	약 57.23MB, 0.75% 감소
평균 peak USS	7658.05 ± 9.00	7607.45 ± 92.8	약 50.60MB, 0.66% 감소
평균 peak GPU mem	17098	17098	변화 없음

실험 결과, CPU 메모리는 50MB 이상 감소했으나, 비율로는 평균 0.7% 정도의 개선에 그쳤으며, CUDA peak memory는 줄어들지 않았다. 이는 본 실험의 peak memory가 Python-level 중간 객체보다 SAM3 내부의 메모리 처리에 의해 지배되었을 가능성을 보여준다.

따라서 본 개선은 SAM3 자체의 GPU memory 사용 구조를 직접 줄인 최적화라기보다, Python 코드에서 제거 가능한 불필요한 중간 결과 저장을 줄인 개선으로 보는 것이 적절하다.

5.3 영상-프롬프트 조합별 병목 파악 가능

최종 수정본에서는 각 영상-프롬프트 조합별 실행 시간과 메모리 사용량이 `prompt_run_metrics.csv`에 자동으로 기록된다. 기존 코드에서는 실패한 조합만 콘솔에 출력되었고, 성공한 조합에 대해서는 실행 시간이나 peak memory가 구조화된 형태로 남지 않았다. 따라서 실험이 끝난 뒤 어떤 영상이나 어떤 프롬프트 조합에서 시간이 오래 걸렸는지, 어떤 조합에서 메모리 사용량이 높았는지 비교하기 어려웠다.

수정 후에는 각 실행 결과가 `PromptRunResult`로 저장되며, 여기에 `elapsed_sec`,

peak_cpu_memory_mb, peak_cuda_memory_mb가 함께 기록된다. 따라서 최종 결과 CSV를 기준으로 영상별, 프롬프트별, 영상-프롬프트 조합별 병목을 확인할 수 있게 되었다.

이 변화는 실행 속도를 직접 줄이는 최적화라기보다는, 실험 결과를 분석 가능한 형태로 남긴다는 점에서 의미가 있다. 기존에는 OOM이 발생하거나 특정 조합의 실행 시간이 길어져도 이를 정량적으로 비교하기 어려웠지만, 수정 후에는 어떤 조합에서 문제가 발생했는지 CSV 기반으로 확인할 수 있다. 특히 여러 영상과 여러 프롬프트를 반복 실행하는 본 실험에서는 전체 평균만 보는 것보다 조합별 결과를 확인하는 것이 중요하다. 해당 정보는 다음과 같이 활용될 수 있다.

- 특정 영상에서만 실행 시간이 길게 나타나는지 확인할 수 있다.
- 특정 프롬프트에서 CUDA peak memory가 높게 나타나는지 확인할 수 있다.
- 실패한 조합이 특정 영상 길이, 해상도, 프롬프트와 관련되는지 확인할 수 있다.
- 이후 평균 실행 시간, 표준편차, 실패율, peak memory 요약 통계를 계산할 수 있다.

다음은 실제 총 세 번 중 첫 번째 실험에서 나온 실행 결과 csv 파일이다. 이를 통해, input1, input2가 다른 두 영상에 비해 cuda 메모리 사용량이 많고, 동시에 실행 시간도 길다는 것을 알 수 있다.

	A	B	C	D	E	F	G	H
1	video_name	prompt	output_path	status	elapsed_sec	peak_cpu_memory_mb	peak_cuda_memory_mb	error
2	input1.mp4	broccoli	results/HD10/tdone		444.5143970000000	22.971442000000000	12408.2421880	
3	input2.mp4	broccoli	results/HD10/tdone		516.7674950000000	17.223539000000000	15646.64892600000	
4	input3.mp4	broccoli	results/HD10/tdone		268.4915970000000	15.692343000000000	9561.35205100000	
5	input4.mp4	broccoli	results/HD10/tdone		328.1985910000000	15.959665000000000	10727.94140600	

5.4 코드 구조 변화

코드 구조 측면에서는 가장 명확한 개선이 있었다. 기존 코드는 run_video_with_prompts() 함수가 대부분의 실행 책임을 가지고 있었다. 이 함수는 입력 영상 프레임 로딩, fps 결정, 출력 경로 생성, skip 여부 확인, SAM3 세션 시작 및 reset, 프롬프트 추가, 예측 결과 수집, 결과 영상 저장, 예외 처리, 실패 목록 반환, CUDA 정리까지 모두 담당했다.

최종 수정본에서는 이러한 책임을 다음과 같이 분리했다.

- BatchRunConfig: 실행 설정값 관리
- VideoPromptJob: 영상-프롬프트 실행 단위 표현
- PromptRunResult: 실행 결과 및 성능 측정값 저장
- Sam3VideoSession: SAM3 세션 start, reset, close 관리
- RunReporter: 진행 상황 출력 및 실패 보고
- run_single_prompt_job(): 단일 영상-프롬프트 조합 실행
- run_video_jobs(): 하나의 영상에 대한 여러 job 실행
- iter_propagated_outputs(): SAM3 output stream generator
- save_masklet_video_streaming(): streaming 기반 결과 영상 저장
- measure_prompt_run: 실행 시간 및 메모리 사용량 측정 decorator
- save_results_csv(): 실행 결과 CSV 저장

이러한 구조 변경으로 인해 전체 코드 길이는 증가했지만, 각 구성 요소의 책임이 명확해져 특

정 기능을 수정할 때 전체 흐름을 모두 파악할 필요가 줄었다. 예를 들어, 세션 관리 정책을 수정하려면 Sam3VideoSession을, 출력 형식을 수정하려면 RunReporter를 보면 된다. 따라서 이를 통해 유지보수성, 재사용성, 확장성이 개선되었다.

5.5 중복 제거 결과

기존 코드에서는 세션 종료와 CUDA 정리 로직이 여러 위치에 반복적으로 등장했다. OOM이 발생한 경우, 일반 예외가 발생한 경우, reset_session에 실패한 경우, 영상 처리가 끝난 경우 모두 세션 종료와 cleanup_cuda() 호출이 필요했기 때문에, 비슷한 코드가 여러 except, finally 블록에 흩어져 있었다.

최종 수정본에서는 이러한 자원 관리 책임을 Sam3VideoSession과 cleanup_cuda() 호출 흐름으로 모았다. 특히 Sam3VideoSession을 context manager로 사용하면서 정상 종료와 예외 발생 종료 모두 같은 해제 경로를 따르도록 했다.

또한 기존 코드에서는 성공, 실패, skip에 대한 출력이 추론 함수 내부와 main 흐름에 흩어져 있었다. 수정 후에는 RunReporter가 출력과 실패 보고를 담당한다. 결과적으로 출력 형식과 보고 로직도 한 곳에서 관리할 수 있게 되었다. 결과는 다음과 같이 정리할 수 있다.

- 세션 종료 코드가 Sam3VideoSession.close()로 집중되었다.
- CUDA 정리 흐름이 cleanup_cuda() 중심으로 정리되었다.
- 실패 결과 관리가 failed_prompts, failed_combis에서 PromptRunResult 리스트로 통합되었다.
- 결과 출력이 추론 함수 내부 print에서 RunReporter로 이동했다.
- output path 생성이 make_output_path()로 분리되었다.
- 실행 시간과 메모리 측정이 measure_prompt_run decorator로 분리되었다.

이러한 구조는 세션 종료와 CUDA 정리처럼 누락될 경우 OOM이나 리소스 누수로 이어질 수 있는 로직을 한 곳에서 관리하여 코드 수정 과정에서의 실수 가능성을 줄인다.

5.6 재사용성 또는 확장성 개선 결과

기존 코드는 parse_args()를 제외한 대부분의 함수가 if __name__ == "__main__" 실행 가드 내부에 정의되어 있었다. 따라서 스크립트를 직접 실행하는 경우에는 문제가 없었지만, 다른 파일에서 함수를 import하여 사용하는 데 어려움이 있었다. 이는 향후 benchmark 코드나 테스트 코드와 연결할 때 재사용성을 떨어트리는 요인이었다.

최종 수정본에서는 helper 함수, dataclass, session class, reporter class, 실행 함수가 모두 모듈 수준에 정의되며, 실행 가드에서는 main()만 호출한다. 이 구조 덕분에 다음과 같은 확장이 가능해졌다.

- benchmark 코드에서 run_video_jobs() 또는 run_single_prompt_job()을 직접 호출할 수

있다.

- 테스트 코드에서 `build_video_prompt_jobs()`, `filter_runnable_jobs()`, `make_output_path()` 등을 독립적으로 검증할 수 있다.
- 다른 실험 스크립트에서 `BatchRunConfig`와 `VideoPromptJob`을 사용하여 동일한 실행 단위를 구성할 수 있다.
- `RunReporter`를 수정하거나 교체하여 콘솔 출력이 아닌 파일 로그, JSON 로그 등으로 확장할 수 있다.
- `measure_prompt_run` decorator를 다른 실행 함수에도 재사용할 수 있다.
- `save_results_csv()`를 통해 실험 결과를 후처리 코드와 쉽게 연결할 수 있다.

즉, 최종 수정본은 단일 실행 스크립트에 가까웠던 기존 코드에서 벗어나, 다른 실험 코드와 결합 가능한 모듈형 코드가 되었다. 이는 연구 코드의 재현성과 확장성 측면에서 중요한 개선이다.

5.7 실험 조건 변경 시 코드 수정 범위 비교

기존 코드는 실험 조건을 변경할 때 `run_video_with_prompts()` 내부를 직접 수정해야 하는 경우가 많았다. 예를 들어 출력 파일명 규칙을 변경하거나, 실패 결과 저장 방식을 바꾸거나, 성능 측정 항목을 추가하려면 같은 함수 내부의 여러 흐름을 함께 확인해야 했다.

최종 수정본에서는 실험 조건별 수정 위치가 더 명확해졌다.

- 출력 파일명 규칙 변경: `make_output_path()`
- 실행 설정 추가: `BatchRunConfig`, `build_config_from_args()`
- 실행 단위 변경: `VideoPromptJob`, `build_video_prompt_jobs()`
- 실행 결과 항목 추가: `PromptRunResult`, `save_results_csv()`
- 세션 reset 정책 변경: `Sam3VideoSession.reset()`
- 로그 출력 형식 변경: `RunReporter`
- 성능 측정 항목 추가: `measure_prompt_run`
- 저장 방식 변경: `save_masklet_video_streaming()`

이를 통해 실험 조건 변경 시 수정해야 하는 코드 범위가 좁아졌다. 연구 과정에서는 프롬프트 목록, 영상 경로, 출력 경로, 측정 지표, 저장 방식이 자주 바뀌기 때문에, 변경 지점이 명확한 구조가 중요하다. 따라서 본 개선은 실행 성능뿐 아니라 실험 조건 변경의 편의성 측면에서도 의미가 있다.

6. 결과 해석 및 한계

6.1 기대한 개선이 실제로 나타난 부분

- **코드 구조가 명확해졌다.** 기존 `run_video_with_prompts()` 중심 구조에서는 하나의 함수가 설정 관리, 세션 관리, 추론, 저장, 예외 처리, 출력 보고까지 모두 담당했다. 수정 후에는

BatchRunConfig, VideoPromptJob, PromptRunResult, Sam3VideoSession, RunReporter 등으로 역할을 분리하여 각 구성 요소의 책임이 명확해졌다.

- **실행 단위가 명시적으로 분리되었다.** 기존에는 영상과 프롬프트 조합이 중첩 반복문 안에서 즉석으로 처리되었다. 수정 후에는 VideoPromptJob을 통해 하나의 영상-프롬프트 조합을 명시적인 실행 단위로 표현했다. 이를 통해 어떤 조합이 실행되었고, 어떤 조합이 skip되었으며, 어떤 조합이 실패했는지 추적하기 쉬워졌다.

- **세션 관리와 CUDA 정리 책임이 분리되었다.** 기존에는 SAM3 session start, reset, close 및 cleanup_cuda() 호출이 추론 함수 내부의 try-except-finally 구조에 직접 들어 있었다. 수정 후에는 Sam3VideoSession context manager로 세션의 생명주기를 관리하도록 하여, 추론 함수가 세션 해제 책임을 직접 갖지 않도록 했다.

- **진행 상황 출력과 실패 보고가 분리되었다.** 기존에는 [DONE], [ERROR], [REPORT] 출력이 추론 흐름 안에 직접 작성되어 있었다. 수정 후에는 RunReporter가 출력과 최종 실패 보고를 담당하도록 분리했다. 이를 통해 추론 함수는 job 실행과 결과 반환에 집중할 수 있게 되었다.

- **성능 측정 가능성이 개선되었다.** 기존 코드는 실패 조합만 콘솔에 출력할 뿐, 실행 시간이나 peak memory를 구조적으로 저장하지 않았다. 수정 후에는 measure_prompt_run decorator를 통해 각 job별 실행 시간, CPU peak memory, CUDA peak memory를 측정하고, PromptRunResult에 부착한 뒤 CSV로 저장할 수 있게 되었다.

- **Python-level eager evaluation 구조가 완화되었다.** 기존 코드는 전체 프레임 리스트와 전체 예측 결과 딕셔너리를 동시에 메모리에 유지했다. 수정 후에는 iter_propagated_outputs()와 save_masklet_video_streaming()을 통해 프레임과 예측 결과를 순차적으로 소비하도록 바꾸었다. 따라서 Python 코드 수준에서 불필요한 중간 객체 저장을 줄일 수 있게 되었다.

6.2 개선이 기대에 미치지 못하는 부분

- **전체 실행 시간은 크게 감소하지 않았다.** 실행 시간의 상당 부분은 SAM3 내부의 predictor.handle_request(), predictor.handle_stream_request()에서 발생한다. 본 개선은 Python 코드 구조와 저장 흐름을 개선한 것이므로, SAM3 모델 자체의 추론 연산량을 직접 줄이지는 않는다.

- **OOM이 발생하는 CUDA의 peak memory 개선 폭은 작다.** 개선 코드는 Streaming 구조를 적용해 전체 프레임 리스트와 output dict를 제거했기 때문에 CPU 메모리 측면에서는 약간의 개선을 보였다. 그러나 CUDA peak memory의 경우, SAM3 내부 로직이 그 사용량을 차지하기 때문에 크게 개선되지 않았다. 따라서 본 개선만으로는 실제 OOM 원인은 해결하지 못했다.

- **실행 시간은 오히려 비슷하거나 일부 증가할 수 있다.** save_masklet_video_streaming()은 프레임 하나씩 읽고, output stream과 frame index를 맞추며, 임시 파일 생성과 ffmpeg 재인코딩까지 수행한다. 따라서 메모리 사용 흐름은 개선되었지만, 저장 과정의 실행 시간은 기존 방식보다 길어질 수 있다.

6.3 개선 과정에서 발생한 trade-off

6.3.1 구조는 명확해졌지만 코드 복잡도가 증가한 부분

최종 수정본은 `run_video_with_prompts()` 중심 구조를 여러 객체와 함수로 분리했다. 이로 인해 각 구성 요소의 책임은 명확해졌지만, 전체 코드 길이와 구성 요소 수는 증가했다.

기존 코드에서는 하나의 큰 함수 안에 대부분의 로직이 들어 있어 위에서 아래로 흐름을 따라가기는 쉬웠다. 반면 수정 후에는 `BatchRunConfig`, `VideoPromptJob`, `PromptRunResult`, `Sam3VideoSession`, `RunReporter`, `measure_prompt_run`, `save_masklet_video_streaming()` 등 여러 구성 요소를 함께 이해해야 한다. 즉, 단순한 스크립트 구조는 줄어들었지만, 유지보수와 확장에 적합한 구조를 얻는 대신 코드 복잡도가 증가했다.

6.3.2 메모리 사용 흐름은 개선했지만 저장 로직이 복잡해진 부분

기존 `save_masklet_video()`는 전체 프레임 리스트와 전체 output dict를 전달하면 결과 영상을 저장할 수 있어 사용 방식이 단순했다. 반면 `save_masklet_video_streaming()`은 원본 영상 프레임과 SAM3 output stream을 동시에 순차 소비하며, 이 과정에서 현재 `frame_idx`와 `next_frame_idx`를 비교하고, output이 있는 프레임과 없는 프레임을 구분해야 한다. 또한 임시 mp4 파일 생성, `cv2.VideoWriter` 사용, `ffmpeg` 재인코딩까지 직접 관리해야 한다. 따라서 전체 프레임과 예측 결과를 한 번에 저장하지 않는 장점은 얻었지만, 저장 함수 내부 구현은 더 복잡해졌다.

6.3.3 성능 측정 기능을 얻었지만 약간의 실행 오버헤드가 생긴 부분

`measure_prompt_run` decorator는 각 job 실행 전후에 `time.perf_counter()`, `tracemalloc`, `torch.cuda.reset_peak_memory_stats()`, `torch.cuda.max_memory_allocated()`를 호출한다. 이를 통해 실행 시간과 메모리 사용량을 정량적으로 기록할 수 있게 되었지만, 측정 자체에도 약간의 비용이 발생한다.

다만 본 코드에서는 SAM3 영상 추론이 상대적으로 무거운 작업이므로, 이 오버헤드는 전체 실행 시간에 비해 크지 않을 것으로 판단된다.

6.3.4 Lazy evaluation을 적용했지만 결과 재사용은 어려워진 부분

Generator 기반 streaming 구조는 한 번만 순차적으로 소비하는 작업에는 적합하다. 결과 영상을 저장하는 현재 목적에서는 전체 데이터를 메모리에 저장하지 않아도 되므로 적절하다.

그러나 generator는 한 번 소비하면 다시 사용할 수 없다. 따라서 같은 예측 결과를 여러 번 탐색하거나, 특정 frame index의 결과를 임의 접근해야 하는 분석 작업에는 불편할 수 있다. 향후 frame별 통계 분석이나 시각화 결과 재사용이 필요하다면 별도의 캐시 또는 결과 저장 구조가 필요하다.

6.4 추가로 개선 가능한 부분

6.4.1 실행 대상 지정 방식 개선

현재 코드는 여전히 `skip_existing` 옵션과 출력 파일 존재 여부에 따라 실행 여부를 결정한다. 따라서 어떤 영상-프롬프트 조합이 실행/제외 대상인지 명시적으로 드러나지 않는다.

향후에는 manifest 파일이나 job list CSV를 입력으로 받아 실행 대상을 명시적으로 지정하는 방식으로 개선할 수 있다. 이렇게 하면 특정 영상과 프롬프트 조합만 재실행하는 실험을 더 직관적으로 관리할 수 있다.

6.4.2 SAM3 내부 메모리 사용 분석

본 개선은 Python 코드의 frame list와 output dict를 제거하는 방식이다. 그러나 CUDA peak memory가 크게 줄지 않는다면, 실제 메모리 병목은 SAM3 내부의 video session state, feature cache, propagation tensor에 있을 가능성이 높다.

따라서 추가 개선을 위해서는 SAM3 내부에서 어떤 tensor가 오래 유지되는지, 어떤 단계에서 peak memory가 발생하는지 분석할 필요가 있다.

6.4.3 성능 측정 기능 옵션화

현재 최종 수정본은 `measure_prompt_run` decorator를 통해 항상 실행 시간과 메모리 사용량을 측정한다. 이는 benchmark에는 유용하지만, 실제 대량 실험에서는 측정 오버헤드를 줄이고 싶을 수 있다.

향후에는 `--measure`와 같은 옵션을 추가하여, 성능 측정이 필요한 경우에만 decorator 기반 측정을 수행하도록 개선할 수 있다.

6.4.4 출력 영상 저장 방식 최적화

현재 `save_masklet_video_streaming()`은 임시 mp4 파일을 만든 뒤 ffmpeg로 재인코딩한다. 이 방식은 기존 SAM3 방식과의 출력 영상 호환성을 유지하는 데 도움이 되지만, 추가적인 I/O와 인코딩 시간을 발생시킨다.

향후에는 ffmpeg pipe를 직접 사용하거나, OpenCV writer 설정을 개선하여 임시 파일 생성을 줄이는 방식도 고려할 수 있다.

6.4.5 benchmark 결과 자동 요약

최종 수정본은 `prompt_run_metrics.csv`를 저장하지만, 평균 실행 시간, 표준편차, 실패율, peak memory 요약 표를 자동으로 생성하지는 않는다.

향후에는 별도의 benchmark 분석 스크립트를 작성하여 CSV를 읽고, 평균 실행 시간, 표준편차, peak memory, 실패율을 자동 계산하도록 개선할 수 있다. 이를 통해 보고서 작성과 실험 결과 비교가 더 쉬워질 수 있다.